

Production Go Codebase Roadmap

A reverse-engineering path for learning large, real-world Go systems

Objective: stop consuming Go tutorials; start extracting production taste from serious infrastructure code.

Core rule: do not read a repo top-to-bottom. Trace runtime flows: startup, write path, read path, failure path, shutdown path.

Roadmap at a Glance

Order	Repository	Primary lesson	Why it matters	Stage
1	etcd	Distributed service internals	gRPC, Raft integration, WAL, snapshots, failure testing	Foundation
2	containerd	Systems daemon architecture	Plugins, runtimes, lifecycle, image/content store, host integration	Runtime systems
3	Caddy	Clean extensible Go architecture	Modules, config, HTTP/TLS, long-running service design	Architecture taste
4	Prometheus	Observability engine internals	Scraping, TSDB, PromQL, service discovery, alerting	Data systems
5	Kubernetes client-go / controller-runtime	Kubernetes API machinery	Informers, caches, workqueues, reconciler patterns	K8s control loops
6	Kubernetes core	Massive production monorepo	Controllers, apiserver, kubelet, generated APIs, historical layers	Boss fight

Repo 1 - etcd

Repo: [etcd-io/etcd](https://github.com/etcd-io/etcd)

Start here. etcd is the best first serious target because it is large, production-grade, and coherent. The whole system exists to solve one hard problem: reliable, strongly consistent cluster state.

What etcd will teach you

- Production Go project structure without Kubernetes-scale chaos.
- gRPC API boundaries, server composition, request validation, and error propagation.
- Raft integration from an engineering perspective: propose, commit, apply, leadership changes.
- Persistence via WAL, snapshots, backend storage, compaction, restore behavior.
- Concurrency discipline: goroutine ownership, contexts, channels, locks, shutdown paths.
- Failure-oriented testing: partitions, leader changes, slow nodes, recovery behavior.

Reverse-engineering path

client request -> gRPC API -> server layer -> raft proposal -> WAL/snapshot/backend -> apply -> response

- Build and run a 3-node local cluster. Kill the leader. Restart nodes. Watch behavior before reading code.
- Trace etcdctl put /foo bar from CLI to client to gRPC to server handler to Raft proposal to apply path.
- Trace etcdctl get /foo and compare serializable vs linearizable read behavior.
- Map startup: config parsing, logger setup, server construction, listener creation, signal handling, graceful shutdown.
- Read tests after reading flows. Tests reveal what maintainers consider correctness.

Deliverables

- request-flow-put.md
- request-flow-get.md
- server-startup-map.md
- raft-integration-notes.md
- storage-wal-snapshot-notes.md
- concurrency-patterns.md
- failure-tests-index.md

Repo 2 - containerd

Repo: [containerd/containerd](#)

Study this after etcd. containerd teaches daemon design, plugin architecture, container lifecycle management, host integration, and the kind of systems Go that lives near Linux.

What containerd will teach you

- Long-running daemon structure and service registration.
- Plugin architecture: how features are discovered, initialized, composed, and isolated.
- Container lifecycle: image pull, content store, snapshots, runtime shim, task creation, supervision.
- Context propagation across APIs and background operations.
- Filesystem and OS-facing Go: namespaces, cgroups, processes, mounts, snapshots.

Reverse-engineering path

ctr image pull -> content fetch -> unpack/snapshot -> container create -> task start -> shim/runtime

- Run containerd locally and use ctr/nerdctl to pull and run a container.
- Trace daemon startup and plugin initialization order.
- Trace image pull into content store and snapshotter interactions.
- Trace container create and task start until runtime handoff.
- Map where containerd stops and runc/runtime/shim begins.

Deliverables

- daemon-startup-plugin-map.md
- image-pull-flow.md
- container-create-task-start-flow.md
- snapshotter-notes.md
- runtime-shim-boundary.md

Repo 3 - Caddy

Repo: [caddyserver/caddy](https://github.com/caddyserver/caddy)

Caddy is the cleaner architecture palate cleanser. It is excellent for learning how to build an extensible Go service with modules, configuration, HTTP/TLS, and runtime reloads.

What Caddy will teach you

- Modular application architecture without turning everything into abstract nonsense.
- Config loading, validation, adaptation, and online reload behavior.
- HTTP server lifecycle, TLS automation, admin APIs, and graceful changes.
- Good boundaries between core platform and pluggable modules.

Reverse-engineering path

Caddyfile/JSON config -> adapter/loader -> module provisioning -> app start -> HTTP/TLS runtime -> reload/shutdown

- Run Caddy with a tiny config, then reload it without restart.
- Trace config parsing and module provisioning.
- Study how modules declare themselves and are wired into the runtime.
- Trace HTTP request handling from listener to handler chain.

Deliverables

- config-load-reload-flow.md
- module-system-map.md
- http-request-flow.md
- graceful-reload-shutdown-notes.md

Repo 4 - Prometheus

Repo: [prometheus/prometheus](https://github.com/prometheus/prometheus)

Prometheus is where you study observability internals: scraping, time-series storage, query execution, service discovery, alerting, and performance-sensitive Go.

What Prometheus will teach you

- TSDB internals: blocks, WAL, compaction, indexing, retention.
- Scrape scheduling, target discovery, relabeling, sample ingestion.
- Query engine structure and PromQL evaluation.
- Production performance tradeoffs around memory, CPU, IO, and cardinality.

Reverse-engineering path

service discovery -> target manager -> scrape loop -> sample ingestion -> TSDB/WAL -> query engine -> API/UI

- Run Prometheus locally with a scrape target.
- Trace how targets are discovered, relabeled, and scheduled.
- Trace one scrape result into ingestion and storage.
- Trace a simple PromQL query through the engine.
- Study TSDB tests for correctness and performance expectations.

Deliverables

- scrape-loop-flow.md
- target-discovery-relabeling.md
- sample-ingestion-tsdb-flow.md
- promql-query-evaluation.md
- cardinality-performance-notes.md

Repo 5 - Kubernetes client-go and controller-runtime

Repos: [kubernetes/client-go](https://github.com/kubernetes/client-go) and [kubernetes-sigs/controller-runtime](https://github.com/kubernetes-sigs/controller-runtime)

This is the bridge into Kubernetes core. Study client-go before Kubernetes itself so the API machinery, watches, informers, caches, and reconciler loops stop looking like dark magic.

What these repos will teach you

- Kubernetes API clients, REST mapping, schemes, serializers, generated clients.
- Watches, informers, shared caches, listers, indexers, and event handlers.
- Workqueues, retries, rate limiting, idempotent reconciliation.
- controller-runtime manager, controllers, reconcilers, admission webhooks, envtest.

Reverse-engineering path

watch event -> informer -> cache/indexer -> event handler -> workqueue -> reconciler -> API write

- Write a tiny controller using controller-runtime.
- Trace how manager starts caches, controllers, leader election, and webhooks.
- Trace an object update from API watch to informer cache to reconcile request.
- Study workqueue retries and rate limiting until you can explain failure behavior clearly.

Deliverables

- informer-cache-workqueue-flow.md
- controller-runtime-manager-map.md
- reconcile-idempotency-notes.md
- api-watch-failure-modes.md

Repo 6 - Kubernetes core

Repo: [kubernetes/kubernetes](https://github.com/kubernetes/kubernetes)

This is the boss fight. Do not begin here. Enter after etcd, containerd, and client-go patterns are already familiar. Kubernetes core is not one codebase; it is a civilization with generated code and battle scars.

What Kubernetes core will teach you

- API server request path, admission chain, storage strategy, watch behavior.
- Controller manager internals and built-in controllers.
- Scheduler framework and scheduling cycle.
- Kubelet sync loops, pod lifecycle, CRI interaction, volumes, probes, status updates.
- Massive monorepo navigation, generated APIs, historical layering, and real-world compromises.

Reverse-engineering path

kubectl apply -> apiserver -> authn/authz/admission -> storage/etcd -> watch -> controller/scheduler/kubelet

- Trace kubectl apply for a Deployment into API server storage.
- Trace Deployment -> ReplicaSet -> Pod controller flow.
- Trace scheduler decision for one pending Pod.
- Trace kubelet pod sync into CRI/container runtime calls.
- Study one controller deeply instead of skimming all controllers badly.

Deliverables

- apiserver-request-path.md
- admission-storage-watch-flow.md
- deployment-replicaset-pod-controller-flow.md
- scheduler-one-pod-flow.md
- kubelet-pod-sync-flow.md

The Method: How to Reverse-Engineer Any Go Codebase

- Start from behavior, not files.
 - Run the binary, send a real request, break it, observe logs and metrics.
- Trace one path at a time.
 - One write path, one read path, one failure path, one shutdown path.
- Map ownership.
 - Which package owns the goroutine? Which package owns cancellation? Which package owns persistence?
- Extract idioms.
 - Error handling, interfaces, package boundaries, context use, testing style, dependency construction.
- Write artifacts.
 - Your notes are the compiler for your understanding. If you cannot draw it, you do not own it yet.

Per-Subsystem Note Template

Package:

Responsibility:

Important structs:

Important interfaces:

Owned goroutines:

Inputs:

Outputs:

Contexts/cancellation:

Persistence/state:

Failure modes:

Tests that prove it:

Open questions:

Suggested 24-Week Timeline

Weeks	Focus	Repo	Outcome
1-4	Distributed service internals	etcd	You can trace write/read/failure/shutdown paths.
5-8	Daemon/runtime internals	containerd	You understand plugins, image pull, container lifecycle, runtime handoff.
9-11	Clean modular service architecture	Caddy	You understand config, modules, reloads, HTTP/TLS lifecycle.
12-15	Observability data system	Prometheus	You understand scrape loop, ingestion, TSDB, query path.
16-19	Kubernetes control-loop machinery	client-go / controller-runtime	You can explain informers, caches, queues, reconcilers.
20-24	Kubernetes core flows	kubernetes/kubernetes	You can trace apiserver/controller/scheduler/kubelet paths.

Final Rule

Do not try to “finish” a codebase. Extract flows, patterns, and failure models. The goal is not completion. The goal is becoming the kind of engineer who can enter an unfamiliar production Go system and calmly find the truth.

Recommended starting command:

```
git clone https://github.com/etcd-io/etcd
cd etcd
# Build it, run it, break it, then trace one write path.
```